Kristiania University College

Security issues in Android Applications

- Understanding Android Apps
- Android security model
- Reverse-engineering
- Exposing vulnerabilities and attacking App components

Introduction

- Android is an **open source** and Linux-based **operating system** for mobile devices.
- It is developed by the Open Handset Alliance (**OHA**) which is led by **Google**.



Introduction ...

- The first commercial version, Android 1.0, was released **in 2008**.
- Android 4.1 (**Jelly Bean**) was released in 2012 with **improved user interface**.
- Source code for Android is available under **free/open source software licenses**.
- Google publishes most of the code under the **Apache License** and Linux kernel **GNU General Public License**.

Android versions

Name	Version number(s)	Initial stable release date	Supported (security fixes)	API level	References
No official codename	1.0	September 23, 2008	No	1	[9]
	1.1	February 9, 2009	No	2	[9][14]
Cupcake	1.5	April 27, 2009	No	3	[15]
Donut	1.6	September 15, 2009	No	4	[16]
Eclair	2.0 – 2.1	October 26, 2009	No	5 – 7	[17]
Froyo	2.2 – 2.2.3	May 20, 2010	No	8	[18]
Gingerbread	2.3 – 2.3.7	December 6, 2010	No	9 – 10	[19]
Honeycomb	3.0 – 3.2.6	February 22, 2011	No	11 – 13	[20]
Ice Cream Sandwich	4.0 – 4.0.4	October 18, 2011	No	14 – 15	[21]
Jelly Bean	4.1 – 4.3.1	July 9, 2012	No	16 – 18	[22]
KitKat	4.4 – 4.4.4	October 31, 2013	No	19 – 20	[23]
Lollipop	5.0 – 5.1.1	November 12, 2014	No	21 – 22	[24]
Marshmallow	6.0 – 6.0.1	October 5, 2015	No	23	[25]
Nougat	7.0 – 7.1.2	August 22, 2016	No	24 – 25	[26][27][28][29]
Oreo	8.0 – 8.1	August 21, 2017	Yes	26 – 27	[30]
Pie	9	August 6, 2018	Yes	28	[31]
Android 10	10	September 3, 2019	Yes	29	[32]
Android 11	11	September 8, 2020	Yes	30	[33]

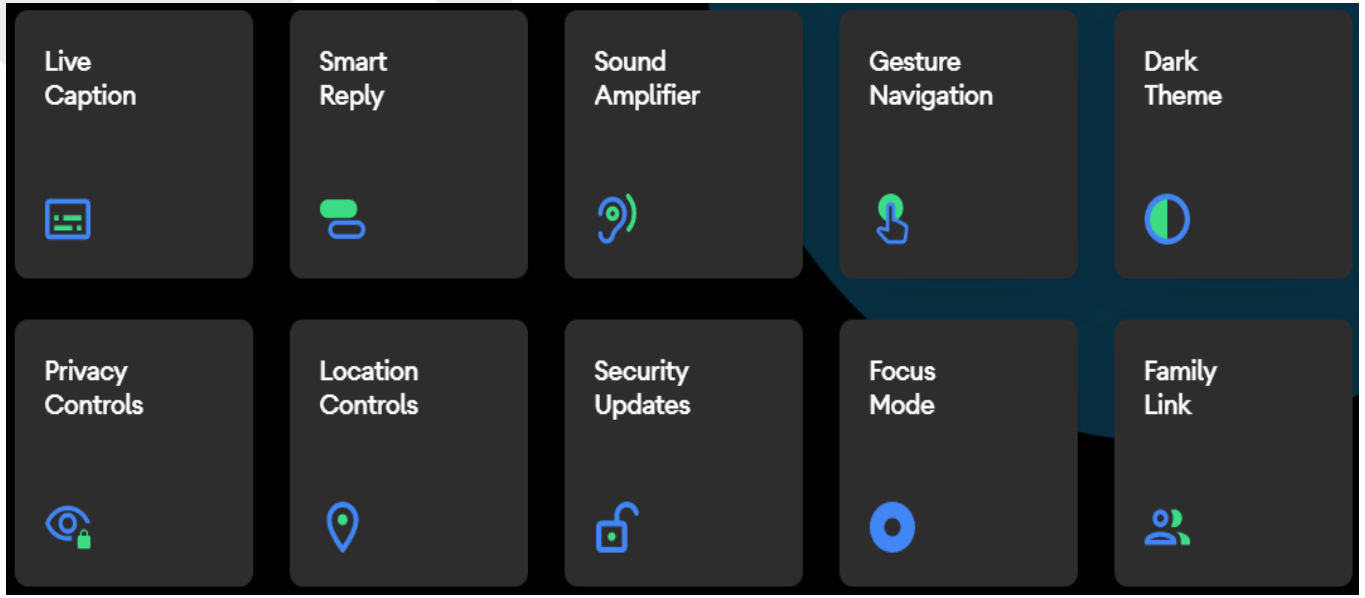
Features of the different Android versions- current transition from version 10 to 11

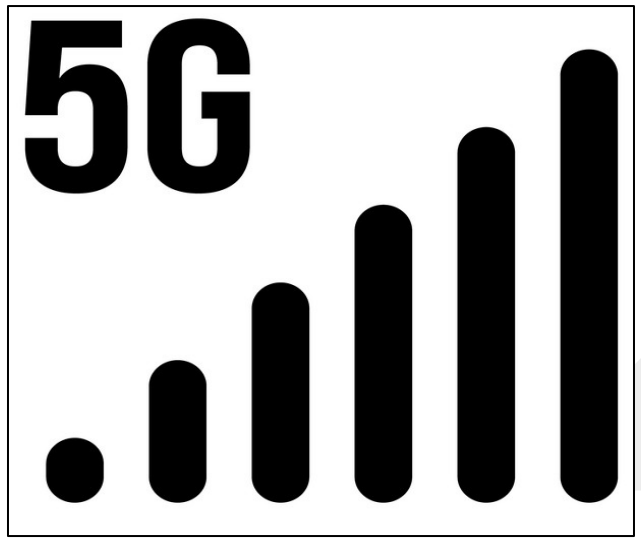
Android 12 was also released on October 4, 2021, which is the latest Android version to date.

Main features of Android

Intuitive user interface	Connectivity (e.g. Wi-Fi)	Storage (e.g. SQLite)
Media support (e.g. MPEG-4)	Messaging (e.g. SMS)	Web browser
Multi-touch	Multi-tasking	Resizable widgets

New features of versions 10/11/12

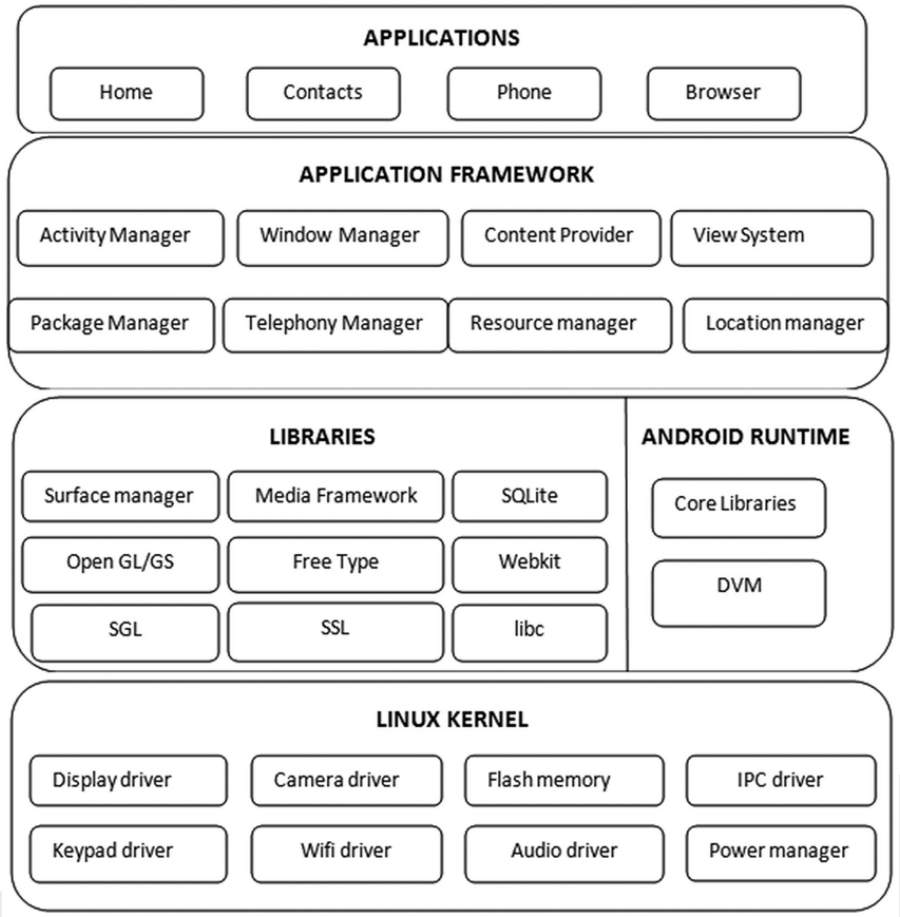






foldable

Application architecture



- Android OS has two distinct features:
  →modified Linux **kernel** (Android kernel)
  →application **virtual machine** that runs Java-like Apps.
  →gap between **mainline Linux kernel** and the Android kernel varied over the years.
- In Linux, applications that are started by a user are **run under that user's context**.
  →Thus, it relies on a user's not **installing malicious software** on their computers …
  →as there is no protection against accessing files that are owned by **same user** …
- In contrast to Linux, however, each App installed on Android is assigned its own **unique** identifier:
  →user identifier (UID) and group identifier (GID).
- Dalvik Virtual Machine (**DVM**) is specifically designed for Android platform and is unique.
- The main reason for the existence of **DVM** is to run on processor and memory limited hardware.
  →So, DVM is lighter than the normal JVM.
- The code that runs on DVM is written and compiled to **Java classes** and converted into a single DEX file.
- Android Apps are distributed in the form of a zipped archive **APK** files which stands for **Android Package**.

Android App Components

Activities- visual screens of an application with which **users interact**. For example, when you launch an application, you see its main activity.

Services- non-graphical components that provide the facility to **perform tasks** that are long running in the **background**.

Broadcast receivers- non-graphical components that allow an application to register for certain system or application **events**.

Content providers- the **data storehouses** of an application that provides a **standard way** to retrieve, modify, and delete data - SQLite.

Android APK folder structure

/Assets - allows the developer to place files in this directory that they would like bundled with the App.

/Res - contains all the App **activity** layouts, images used, and any other files that the developer would like accessed from code in a structured way.

/Lib - contains any **native libraries** that are bundled within the App- loaded by the application according to the detected CPU architecture- like x86, ARM, MIPS.

/META-INF - this folder contains the **certificate** of the App and files that hold an inventory list of all included files in the zip archive and their hashes.

/classes.dex - this is the **executable** file containing the Dalvik bytecode of the application. It is the actual code that will run on the Dalvik Virtual Machine.

/AndroidManifest.xml - the manifest file containing all configuration information about the application and defined **security parameters**.

/Resources.asrc - resources can be compiled into this file instead of being put into the res folder- contains any **application strings**.

Android SDK tools

- Android Apps are usually developed in **Java** language using Android Software Development Kit (SDK)- requires **downloading** the Android **SDK**.
- Exploit the Android Developer Tools package → integrated development environment (IDE) and all the tools.
- Whether you plan to use an **emulator** or **physical** device, the Android SDK provides many tools that are essential **for Android hacking**.
- **Emulator** is among the Android SDK tools which provide root access by default.
- However, emulators do not use hardware like USB, headset, **Wi-Fi**, **Bluetooth**, **real phone calls**, NFC, SD card, etc.
- At the lowest levels of operation, an emulator is **not an exact replica** of how Android runs on a real device.
- Real Android devices do not provide root access by default. However, significant testing/hacking can be performed **without having root access** on an Android device.

Android SDK tools …

Should I Use a Simulator/Emulator or a Real Device When Developing/Testing Mobile Applications?		
	SIMULATOR/EMULATOR	REAL DEVICE
I am doing a mock-up of my application	✔	
I need to debug my application	✔	✔
I want to test my application code	✔	
I need to test end-to-end flows of my application		✔
I need to verify the networking aspects of my app		✔
I want to verify my app performance (CPU/Memory)		✔
I want to know what happens when my app is interrupted by notifications or phone calls		✔
I want to verify the camera functionality within my app		✔
I need to verify app graphics and transitions	✔	✔
I need to verify the app from an end user perspective, such as testing screen brightness, resolution, etc.		✔
I need to make sure my app runs on various devices and OS versions		✔
I want to run regression tests against my app		✔
I want to test the battery life, or how fast the battery drains, based on usage of my app		✔
I need to test geolocation and screen/app orientation (accelerometer) functionality		✔

https://mobilelabsinc.com/blog/simulators-vs-emulators

Kristiania University College

Android SDK tools …

- **adb** is used to explore **devices/emulators-** install apps, run shell, read logs, forward ports, show devices, push file to, etc.
- **drozer** is Android pen testing tool that finds vulnerabilities in apps or devices and provides exploits, useful payloads.
- **aapt** (Android Asset Packaging Tool) is used to packages assets into binary and converts binary to readable text to perform reverse-engineering.

Android security model

- The foundation of the Android app security model is that **no two apps** running on the same device should be able to **access each other's data** without authorization.
- They should also not be able to **affect the operation** of the other app adversely or without appropriate **consent** which is the basis of an application **sandbox**.
- An app would need to know if another **app is authorized** to perform an action, hence application's **identity** is important.
- **Code signing** is done on Android package through digital certificates where the private key is only held by developers to **prove identity** of an app's author thereby get trust.
- Tools: **$keytool –genkey**, **$jarsigner … application.apk alias_name**- also use in Windows command **ssh-keygen -t rsa**



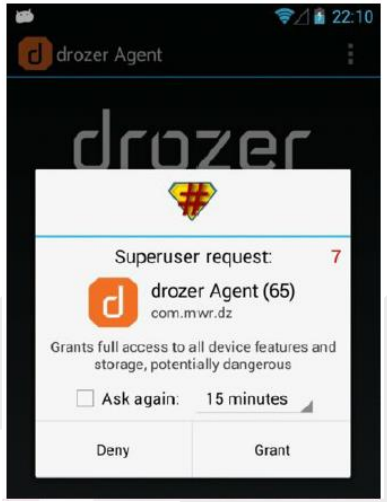
Kristiania University College

Android security model …

- App certificates are **not verified by the Android** operating system in any way and do not need to be issued by a certain Certificate Authority (CA) like other platforms.
- The signing certificate is checked only **when the application gets installed** and if the certificate subsequently expires, the application will still run as normal.
- Google recommends that signing certificates be created with a **validity period** of 25 years or longer to support seamless updates to your application.
- A number of vulnerabilities have been **discovered** in the way that the validation of signatures is performed on APK files.
- Android employs a fine-grained **privilege model** for applications to request **permission** to access resources.

Android security model …

- In drozer, permissions that have been requested **can be accessed** using
  dz> run app.package.info -a com.android.browser.
- File system (full disk) encryption (**FDE**) is when the **entire drive are encrypted-** available on Android since version 3.0 (Honeycomb).
- FDE serves as **protection against stolen or lost disks-** it uses **dm-crypt** module in kernel to encrypt and decrypt data on the block device layer (see video here).
- Android does not allow Apps to run tasks as a **root** user (super user permission) which led researchers to find ways to obtain root on Android devices, hence **rooting**.
- A typical **objective of rooting** a device is to put the *su command* in a directory on the PATH (e.g., /system/bin or /system/xbin).



An example of a **prompt** from the SuperSU application

Kristiania University College

Reverse-engineering Apps

- Reverse-engineering is the process of gaining a **deep understanding** of a finished product.
- In Android, first, **find the package** of a target App:

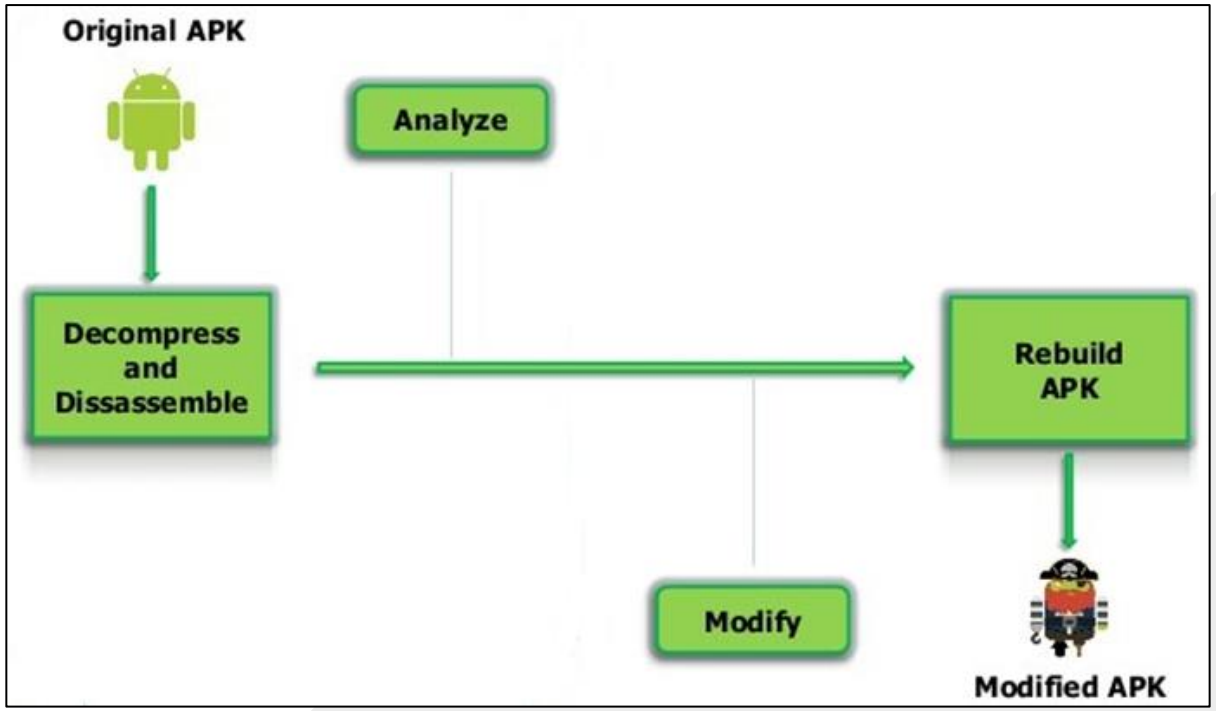
  $adb shell pm list packages | grep app-name
- To pull the App off the device you need to find the APK path:

  $adb shell pm path package-name
- To **reverse engineer** an App from Play Store, you need to install it on your device **or** get APK by sending package name to the sites.
- A **DEX** (dalvik executable) is an executable file saved in a format that contains compiled code written for Android.
- DEX files can be **disassembled** into human-readable assembly from adb shell:

  $dexdump -d /path/to/classes.dex

Reverse-engineering Apps …

- **Apktool** is used to reverse-engineer an entire Android package back for modification.
- IDA is another popular disassembler used by reverse engineers all around the world.
- Application analysis workflow in Android:



Exposing vulnerabilities

- App components to consider are **App container, communications and Internet server**.

- **Exported components** (APIs) App can be found and exposed by inspecting its manifest using the below **drozer** command:

  dz> run app.package.attacksurface com.android.browser
  Attack Surface:
  6 activities exported
  4 broadcast receivers exported
  1 content providers exported
  0 services exported

- The **root** and **system users** can exert control over installed Apps and **can interact** with components even when they are not exported.

- Whether an App exports a component in its manifest or not, it is **relevant only when** the calling App is another **non-system App**.

Kristiania University College

Exposing vulnerabilities … attack scenario

An installed **malicious App defines a set of known permission** names from legitimate App with a protection level of **normal**.

User installs legitimate App, and OS sees that one of the permissions is already defined. This leads **the OS to ignore the protection** level and stick to the known parameters defined by the malicious App.

The permission now has a downgraded protection level of **normal** instead of more secure value (e.g., signature). Android does not know any difference even if the permission of the legitimate App was defined with signature.

The **malicious App can interact with the no-longer protected** legitimate App components defined with the downgraded permission.

Attacking App components

- Attacking an app over the Android IPC system **involves finding all the exported** components of the App and attempting to use it in a way that was not intended.
- For activities, broadcast receivers, and services, this means you must examine all **the code that handles intents** from other Apps.
- In Android, an **intent** is a data object that loosely defines a task to be performed and it can be matched against **intent filters** based on its action, data, and category.

- Common Android vulnerabilities can be showcased using Apps like **Sieve, password manager App**.
- After installing it, you can find the package name of Sieve by using **dz> run app.package.list -f Sieve**
- **Examine exported App components** of Sieve in its manifest using tools like drozer.

Attacking App components …

Insecure storage:

- The **sandbox** provides segregation of Apps' data mainly based on file/folder ownership and permissions, i.e., **each file and folder belongs** to an owner and a group.
- However, the owner/group of a file **can be changed as the root user using a tools** such as **chown** and **chmod**.

Insecure communication:

- Apps are often designed to make use of **HTTP/HTTPS** in order to easily integrate into existing infrastructure to **send and receive information** from services on the Internet.
- However, **native apps are often less secure** than web browsers which may also make use of **other** communication **protocols**.
- Thus, communication flaws of Apps can be **misused** to attack the device.

Top Privacy Changes in Android 10

	Privacy change	Apps affected	Mitigation strategy
✓	**Scoped storage** Filtered view into external storage, giving access to app-specific files and media collections	Apps that access and share files in external storage	Work in app-specific directory and media collection directories Learn more
✓	**More user control over location permissions** Foreground-only permission that gives users more control over app access to device location	Apps that request the user's location while in the background	Ensure graceful degradation in the absence of background location updates Use permission introduced in Android 10 to access location in the background Learn more
✓	**Background activity starts** Restrictions on launching activities from the background	Apps that launch activities without user interaction	Use notification-triggered activities Learn more
✓	**Non-resettable hardware identifiers** Restrictions on accessing device serial and IMEI	Apps that access device serial or IMEI	Use an identifier that the user can reset Learn more
✓	**Permission for wireless scanning** Access to some Wi-Fi, Wi-Fi Aware, and Bluetooth scanning methods requires fine location permission	Apps using Wi-Fi and Bluetooth APIs	Request `ACCESS_FINE_LOCATION` permission for related use cases Learn more

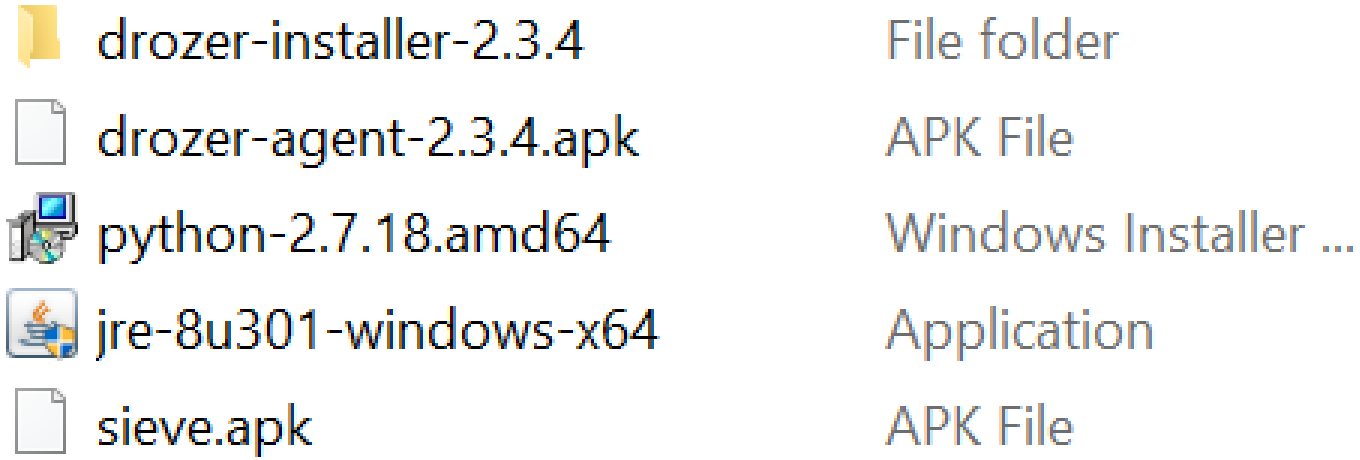
Top Privacy Changes in Android 11

	Privacy change	Apps affected	Mitigation strategy
✓	**Scoped storage enforcement** Apps that target Android 11 or higher are always subject to scoped storage behaviors	Apps that target Android 11 or higher, as well as apps that target Android 10 and haven't set `requestLegacyExternalStorage` to `true` to opt out of scoped storage	Update your app to work with scoped storage Learn more about the scoped storage changes
✓	**One-time permissions** Users can grant temporary access to location, microphone, and camera using one-time permissions	Apps that run on Android 11 or higher and request location, microphone, or camera permissions	Check that your app has a permission before attempting to access data that's guarded by that permission Follow best practices for requesting permissions
✓	**Permissions auto-reset** If users haven't interacted with an app for a few months on Android 11 or higher, the system auto-resets the app's sensitive permissions	Apps that target Android 11 or higher and perform most of their work in the background	Ask the user to prevent the system from resetting your app's permissions Learn more about permissions auto-reset
✓	**Background location access** Android 11 changes how users can grant the background location permission to apps	Apps that target Android 11 or higher and need access to background location	Request foreground (coarse or fine) and background location permissions incrementally in separate calls to the permission request method. When necessary, explain the benefits that users receive for granting that permission Learn more about background location access in Android 11
✓	**Package visibility** Android 11 changes how apps query and interact with other installed apps on the same device	Apps that target Android 11 or higher and interact with other installed apps on a device	Add the `<queries>` element to your app's manifest Learn more about package visibility
✓	**Foreground services** Android 11 changes how foreground services can access location, camera, and microphone data	Apps that run on Android 11 or higher and access location, the camera, or the microphone in a foreground service	Declare the `camera` and `microphone` foreground service types for the foreground services that require access to the camera and microphone, respectively. Be aware, however, that foreground services that start while the app is in the background usually cannot access location, camera, or microphone. Learn more about the changes to foreground services

Kristiania University College

Demo

To set up drozer use the following tools:



Demo …

Install drozer-agent software from adb

>adb –s device install path/drozer-agent-2.3.4.apk

>adb -s device install path/sieve.apk

>adb uninstall com.mwr.example.sieve

Starting the drozer agent runs on port 31415

Port forward to establish the communication between the Drozer Client and Agent:

>adb forward tcp:31415 tcp:31415

Finally, launch the application and press the bottom "ON“. And connect to it:

drozer console connect

To find installed app's package name

dz> run app.package.list -f Sieve

To get Manifest of the app

dz> run app.package.manifest com.mwr.example.sieve

To explore the attack surface of the App

dz> run app.package.attacksurface com.mwr.example.sieve

Examine exported activities- permissions

dz> run app.activity.info -a com.mwr.example.sieve

dz> run app.package.info -a com.android.chrome

More information on basic drozer commands can be found in link 1 and link 2.

Class activities:

1. Individual participation 1:
   Discussion questions via Mentimeter (Week42)
2. Individual participation 2:
   Questions via Kahoot (Week42)
3. Group activity:
   List of activities to be provided
4. Work on “Obligatory assignment 2“

NB: Additional information about the activities yet to be provided in separate document.

Next topics

Exploiting vulnerabilities and writing Secure Android Apps